

Chris Stevens

Professor Neil Kalinowski

CS-499

30 July 2026

Milestone 4 Narrative

The CS-340 Grazioso Salvare Animal Shelter artifact created in June 2025 is used for all three enhancement categories. For this milestone, I focused on the database changes needed to support user authentication, authorization, and controlled access to animal records. Earlier enhancements had already moved the project to a Docker-based MongoDB deployment, added API-based CRUD operations, and added the rescue profile collection used by the scoring algorithm. This milestone completed the remaining database work by replacing the hardcoded login with database-backed user records.

Before this enhancement, session handling was in place, but the user was not being looked up from the database. There was a login service stub that used static login credentials. To complete the login service functionality, I created an `app_users` collection that stores user email, password hash, active status, roles, and shelter claims. The login process now queries MongoDB for an active user and validates the entered password against the stored hash. Beyond authentication, the database enhancement also implemented authorization by including roles and shelter claims, which allows users to be associated with specific shelter records. An admin can work across the full dataset, while a shelter admin can be limited to the records they are allowed to manage. The lowest access role, user, can read the animal database.

The following course outcomes are supported by these changes:

- Develop a security mindset that anticipates adversarial exploits in software architecture and designs to expose potential vulnerabilities, mitigate design flaws, and ensure privacy and enhanced security of data and resources.

The application no longer relies on a hardcoded login account. User records are stored in MongoDB, passwords are stored as hashes, and the login process validates the entered password against the stored hash. Role checks and shelter claims also reduce the risk of users accessing or modifying records they should not manage.

- Demonstrate an ability to use well-founded and innovative techniques, skills, and tools in computing practices for the purpose of implementing computer solutions that deliver value and accomplish industry-specific goals.

The database enhancement supports this outcome by making the application more complete and maintainable. The project now uses MongoDB collections for animal records, rescue profiles, and application users. The Docker seed process loads the application data needed for the system to run, and the Node/Express API uses Mongoose models to interact with the database. This supports the application as a full system instead of only a dashboard or static demo.

- Employ strategies for building collaborative environments that enable diverse audiences to support organizational decision making in the field of computer science.

The database changes help support different users working in the same application. Shelter users can maintain animal records, while rescue users can review animal data and candidate rankings. Adding user roles and shelter claims gives the application a better structure for supporting different responsibilities. This is important because the application supports multiple audiences, but those users should not all have the same level of access.

Some of the database work was already completed during the first two enhancements. The software engineering enhancement required the animal data to be loaded into MongoDB through Docker and exposed through API-based CRUD operations. The algorithms enhancement added rescue profile documents so scoring rules could be stored in MongoDB and updated without changing code. Because of that, this milestone focused on completing the remaining database design needed for authentication and authorization.

A challenge during this milestone was selecting the best approach for password hashing. I originally started with Node's built-in crypto library, but the API was more complex than I needed for this project. After researching other options, I switched to bcrypt, which provided a simpler pattern for hashing passwords and comparing login attempts. This made the authentication code easier to understand and maintain while still supporting secure password storage.

This enhancement helped me better understand how database design supports application security. The way the user record is structured affects login, permissions, API access, and which records a user is allowed to manage. Adding seeded user records makes the application easier to test because the authentication flow can be recreated consistently when the containers are rebuilt. With this change, MongoDB supports the core application data, scoring profiles, and user access records.